

PhishGuard-AI

The complete technical course

How the platform works, from the PostgreSQL database and the Flask/Python backend to the Vue.js interface — every technology, why it was chosen, how the pieces talk to each other, and how to explain every page and every result.

Written for a developer who knows JavaScript, React, Django, Laravel and Tailwind, and is meeting Python, Flask, Vue.js and PostgreSQL through this project.

Cameroon · September 2026

Contents

Contents	2
0. How to use this course	4
1. The big picture	5
1.1 What PhishGuard-AI does	5
1.2 The three layers	5
1.3 How a request travels	5
1.4 Folder map	5
2. Translation dictionary: from your stack to this one	7
3. Python for a JavaScript developer	8
3.1 Side by side	8
3.2 The five Python features you will see everywhere in this project	8
4. The backend: Flask	9
4.1 Why Flask (and not Django)	9
4.2 The application factory: create_app()	9
4.3 Blueprints (the route files)	9
4.4 Anatomy of an endpoint	10
4.5 The response envelope	10
4.6 Services: one job per file	10
5. The data layer: PostgreSQL and SQLAlchemy	11
5.1 Why PostgreSQL	11
5.2 The tables	11
5.3 Models: tables as Python classes	11
5.4 Migrations the PhishGuard way	12
6. The detection engine (the heart of the product)	13
6.1 Parsing (parser.py)	13
6.2 Signal A — the ML model (30 %)	13
6.3 Signal B — Claude AI (25 %)	13
6.4 Signal C — URL intelligence (25 %)	13
6.5 Signal D — rules, sender and headers (20 %)	14
6.6 Fusion (fusion.py)	14
6.7 Languages	14
7. Security architecture	15
7.1 How a login works	15
8. The admin Security Operations Centre (SOC)	16
8.1 Report triage (80 % automation)	16

8.2 Incidents (campaign correlation)	16
8.3 Threat intelligence, allowlist, audit, system	16
8.4 Background jobs (scheduler.py)	16
9. Mailbox scanning, forwarding and "analyse first, sign in to see"	17
9.1 Connecting Gmail / Outlook (OAuth 2.0)	17
9.2 "Forward it to PhishGuard"	17
9.3 Visitor gating	17
9.4 Encrypting tokens (crypto_box.py)	17
10. The frontend: Vue.js from a React developer's point of view	18
10.1 Why Vue 3 + Vite + TypeScript + Tailwind	18
10.2 A Single File Component, read like React	18
10.3 The layers of the frontend	18
10.4 A store, read like Redux/Zustand	19
10.5 Installable app (PWA)	19
11. Page by page: what you see and where it comes from	20
11.1 Public pages	20
11.2 The checker, component by component	20
11.3 User space	20
11.4 Admin console (/admin)	21
12. Three journeys, end to end	22
12.1 A visitor checks an SMS, then signs in	22
12.2 A phishing email arrives in a connected Gmail	22
12.3 An admin deletes a user (sudo mode)	22
13. Reading a result: every output explained	23
14. Running, testing and deploying	24
14.1 Daily commands	24
14.2 Configuration (.env)	24
14.3 Going to production (what you would say to a jury)	24
15. Questions a jury may ask — and how to answer	25
16. Glossary	26

0. How to use this course

This document is written like a course. Each module builds on the previous one. You do not need to have written Python or Vue before: every new idea is first compared with something you already know (React, Django, Laravel, Tailwind, plain JavaScript), then shown with real code taken from PhishGuard-AI.

- **Modules 1–2** give you the big picture and a "translation dictionary" from your stack to this one.
- **Modules 3–5** teach Python, Flask and PostgreSQL through the backend.
- **Modules 6–9** explain the heart of the product: the detection engine, security, the SOC and the mailbox scanner.
- **Modules 10–11** teach Vue.js through the frontend, then walk through every page and what it displays.
- **Modules 12–13** follow real requests end to end and explain every number a result shows.
- **Modules 14–16** cover running/testing, the questions a jury is likely to ask, and a glossary.

Tip for your defence. When you explain a feature, always follow the same path: *what the user sees* (page/component) → *which request is sent* (service → endpoint) → *what the server does* (blueprint → service → pipeline) → *what is stored* (table) → *what comes back* (JSON) → *how it is displayed*. Module 12 does exactly this three times.

1. The big picture

1.1 What PhishGuard-AI does

PhishGuard-AI protects Cameroonian citizens against phishing: fake MTN MoMo / Orange Money SMS, fake bank emails, WhatsApp "fake relative" scams, fake job offers. A user pastes a message (or a screenshot, an .eml file, a shared message, a forwarded email, or connects a whole mailbox). The platform answers in a few seconds with a clear verdict (dangerous / suspicious / safe), the reasons, and what to do. Administrators run a small Security Operations Centre (SOC): they review reports, see scam campaigns grouped into incidents, block domains, and monitor the system.

1.2 The three layers

Layer	Technology	Role	Folder
Presentation (frontend)	Vue 3 + TypeScript, Vite, Tailwind CSS, Pinia, Vue Router	Everything the user sees. Runs in the browser. Talks to the backend only through HTTP/JSON.	frontend/PhishGuard-AI/
Application (backend)	Python 3.13, Flask 2.3, SQLAlchemy, Flask-Login	Business logic: authentication, analysis pipeline, SOC automation, APIs. Runs on port 5000.	backend/app/
Data	PostgreSQL 18	Stores users, analyses, incidents, settings, audit trail, mailbox connections.	backend/database/scripts/
External services	Anthropic Claude API, OpenPhish feed, Gmail/Microsoft Graph APIs, SMTP/IMAP, ip-api.com	AI analysis, known phishing URLs, mailbox access, emails, IP geolocation.	(called from backend services)

1.3 How a request travels

```
Browser (Vue page)                                Vite dev server :5173                                Flask :5000                                PostgreSQL
-----
component -> store -> service ---> proxy "/api/*" -----> before_request hooks
(Analyzer)  (Pinia)  (apiFetch)                                -> blueprint route (scan.py)
                                                    -> services / pipeline ---> SELECT
                                                    <- dict                                <---
component <- store <- service <--- JSON {data, meta} <----- after_request wraps JSON
```

In development two servers run side by side: Vite serves the Vue app on port 5173 and forwards every URL that starts with /api to Flask on port 5000 (see vite.config.ts, key server.proxy). Because the browser only ever talks to 5173, cookies and sessions work without CORS headaches — the same trick as Laravel Mix/Vite proxy or the React "proxy" field in package.json.

1.4 Folder map

```
PhishGuard/
├── backend/
│   ├── run.py                # starts Flask (like "php artisan serve" / "manage.py runserver")
│   ├── .env                  # secrets (never committed)
│   ├── requirements.txt      # Python dependencies (like package.json / composer.json)
│   ├── database/scripts/    # 01_schema.sql ... 09_user_experience.sql (SQL migrations)
│   └── app/
│       ├── __init__.py      # create_app(): config, extensions, hooks, blueprints
│       ├── models.py        # SQLAlchemy models (tables as Python classes)
│       ├── api/              # blueprints = route files (controllers)
│       ├── services/         # business logic, one job per file
│       ├── pipeline/         # the phishing analysis pipeline (v2)
│       └── ml_engine/        # machine-learning models and training scripts
```

- ■■■ templates/email/ # logo used in HTML emails
- frontend/PhishGuard-AI/
 - index.html, vite.config.ts, package.json
 - public/ # favicon, manifest.webmanifest, sw.js (installable app)
 - src/
 - main.ts, App.vue # entry point and root component
 - router/ # URL -> page
 - views/ # pages (public, user, admin)
 - components/ # reusable pieces (check/, admin/, auth/, dashboard/, ui/)
 - stores/ # Pinia state (auth, analysis, notifications...)
 - services/ # one file per API domain, all built on http.ts
 - composables/ # reusable logic (= React custom hooks)
 - i18n/ # translations fr / en / pcm / ewo / ff
 - types/, utils/, data/

2. Translation dictionary: from your stack to this one

Almost every idea in this project exists in a tool you already know. Keep this table open while reading the code.

You know...	In PhishGuard it is...	Comment
Django / Laravel project	Flask app built by create_app()	Flask is a micro-framework: you add only what you need (ORM, login, CORS) as extensions.
Django app / Laravel route group	Blueprint (app/api/*.py)	A group of routes with a URL prefix, e.g. admin_bp mounted on /api/admin.
Route + controller method	@bp.route(...) above a Python function	The function returns a dict / JSON. Decorators play the role of middleware on one route.
Laravel middleware / Django middleware	before_request / after_request hooks, decorators like @login_required	Global hooks in app/__init__.py; per-route decorators in api/security.py.
Eloquent / Django ORM	SQLAlchemy (Flask-SQLAlchemy)	User.query.filter_by(email=x).first() ≈ User::where("email", x)->first()
Migrations (artisan migrate)	Numbered SQL scripts 01...09	Run in pgAdmin as postgres. Model classes are updated by hand to match.
MySQL	PostgreSQL	Same SQL, plus JSON columns, better concurrency, SERIAL ids, stricter types.
.env + config	.env read with python-dotenv, os.getenv()	Identical idea.
composer / pip / npm	pip + requirements.txt, venv	venv = project-local node_modules for Python.
Sanctum / Django sessions	Flask-Login session cookie	No JWT: an HttpOnly cookie identifies the user.
React component (JSX)	Vue Single File Component (.vue)	<script setup> + <template> + <style> in one file.
useState / useMemo / useEffect	ref() / computed() / watch() , onMounted()	Vue tracks dependencies automatically: no dependency arrays.
Custom hooks (useX)	Composables (src/composables/useX.ts)	Same pattern, same naming.
Redux / Zustand / Context	Pinia stores	A store = state + getters + actions, used with useXStore().
React Router	Vue Router	Routes array + beforeEach guard for authentication.
axios / fetch wrapper	apiFetch() in services/http.ts	Adds language header, timeout, error normalisation, sudo retry.
{cond && <X/>}, list.map()	v-if , v-for	Directives written as HTML attributes.
value + onChange	v-model	Two-way binding in one attribute.
props + callback props	defineProps + defineEmits	Children emit events; parents listen with @event.
Tailwind	Tailwind CSS 4	Exactly the same utility classes; dark mode via a .dark class on .
Blade / Django templates for emails	Python f-string HTML in services/email_templates.py	Inline styles because email clients strip CSS.
Celery / Laravel queues + scheduler	services/scheduler.py (a background thread)	Runs periodic jobs: mailbox sync, correlation, feed refresh...

3. Python for a JavaScript developer

Python reads like pseudo-code. The main differences from JavaScript are: indentation instead of braces, no semicolons, None instead of null, True/False capitalised, and and / or / not instead of && || !.

3.1 Side by side

<pre>// JavaScript const name = "Awa" const msg = `Hello \${name}` if (score > 80 && !blocked) { ... } const obj = { a: 1 }; obj.a obj.b ?? 0 const arr = [1, 2]; arr.length arr.map(x => x * 2) arr.filter(x => x > 1) function add(a, b = 1) { return a + b } try { } catch (e) { } finally { } import x from "./x" null / undefined</pre>	<pre># Python name = "Awa" msg = f"Hello {name}" # f-string if score > 80 and not blocked: ... # indentation = block obj = {"a": 1}; obj["a"] # dict obj.get("b", 0) arr = [1, 2]; len(arr) [x * 2 for x in arr] # list comprehension [x for x in arr if x > 1] def add(a, b=1): return a + b try: ... except Exception as e: ... finally: ... from app.services import scan_service None</pre>
---	--

3.2 The five Python features you will see everywhere in this project

Decorators (@something)

A decorator wraps a function to add behaviour — exactly like a middleware on one route.

@login_required refuses the request if no user is signed in; @sudo_required (our own, in api/security.py) asks for the password again.

```
@user_dashboard_bp.route('/summary', methods=['GET']) # URL: GET /api/user/summary
@login_required                                       # "middleware": must be signed in
def monthly_summary():
    ...
    return ok({...})                                # JSON response
```

Context managers (with ...)

with opens a resource and guarantees it is closed, even on error — like try/finally. Used for SMTP connections, IMAP, thread pools and Flask application contexts.

```
with smtplib.SMTP(host, 587, timeout=10) as smtp: # connection closed automatically
    smtp.starttls(); smtp.login(user, password); smtp.send_message(message)
```

Type hints

def claim(token: str, user_id: int) -> Analysis | None — like TypeScript annotations. Python does not enforce them at runtime; they document the code and help the editor.

Modules and packages

Every .py file is a module; a folder with __init__.py is a package. app/__init__.py is therefore the "index" of the app package — this is where create_app() lives.

Virtual environment

backend/venv contains the project's own Python and libraries (like node_modules). Activate it with venv\Scripts\activate, then pip install -r requirements.txt.

4. The backend: Flask

4.1 Why Flask (and not Django)

- The core of the project is an **API** + an **analysis pipeline**, not server-rendered pages: Flask is small and lets us shape exactly that, without Django's admin/templates/forms we would not use.
- Python is the language of machine learning (scikit-learn, NLTK) and of the Anthropic SDK: the ML models and the web server share one process.
- Extensions give us exactly what we need: Flask-SQLAlchemy (ORM), Flask-Login (sessions), Flask-CORS, Flask-Migrate.

4.2 The application factory: create_app()

File backend/app/__init__.py. run.py calls create_app(), which builds and returns the configured app. Step by step it:

- reads configuration (SECRET_KEY, DATABASE_URL, cookie settings) from .env;
- initialises extensions: **db** (SQLAlchemy), **migrate**, **CORS** (allowed frontend origins, credentials), **login_manager**;
- registers **before_request** hooks: request id + timer, start of the background scheduler, idle-session timeout;
- registers **after_request** hooks: wraps every /api JSON in the standard envelope, records timing for the System page, logs slow requests;
- defines **load_user()**: turns the session cookie into a User (and refuses outdated / suspended / unapproved sessions);
- registers every **blueprint** under its URL prefix.

4.3 Blueprints (the route files)

Blueprint file	Prefix	What it serves
api/routes.py	/api	health check, legacy analyse endpoint, threat categories, threat-intel map data
api/auth.py	/api/auth	register (risk-scored), login, logout, me, MFA setup / verify / disable, email codes
api/scan.py	/api/v2	POST /scan (text or .eml), /scan/claim, /scan/<id>, /scan/<id>/feedback
api/user_dashboard.py	/api/user	overview KPIs, history, notifications, report, summary, quiz, delete history
api/user_profile.py	/api/user	profile, avatar, password change
api/mailbox.py	/api/mailbox	connect Gmail/Outlook, OAuth callback, sync, pause, disconnect
api/public.py	/api/public	community scam alerts (anonymised)
api/admin.py	/api/admin	users + approval workflow, platform settings
api/admin_ops.py	/api/admin	review queue + triage, allowlist (whitelist) + domain health
api/admin_security.py	/api/admin/security	security posture, sudo, sessions, audit events
api/admin_soc.py	/api/admin	system metrics, incidents, threat intel, blocklist
api/oauth.py, otp.py, verification.py, mobilemoney.py	various	Google login, OTP, email verification, mobile-money helpers

4.4 Anatomy of an endpoint

```
@scan_bp.route('/scan/<int:analysis_id>/feedback', methods=['POST']) # URL + method
@login_required # must be signed in
def feedback(analysis_id: int): # <int:...> -> argument
    analysis = Analysis.query.filter_by(id=analysis_id, # ownership check:
                                       user_id=current_user.id).first() # only YOUR analysis

    if analysis is None:
        return err('NOT_FOUND', tr(request_lang(), 'error.not_found'), 404)
    body = request.get_json(silent=True) or {} # JSON body (like req.body)
    if body.get('value') not in (1, -1):
        return err('VALIDATION_ERROR', 'value must be 1 or -1', 400) # validate input
    analysis.feedback = body['value']
    db.session.commit() # save (like $model->save())
    return ok({'id': analysis.id, 'feedback': analysis.feedback}) # {data, meta}
```

4.5 The response envelope

Every API answer has the same shape, produced by `ok()` / `err()` (`api/responses.py`) or by the `after_request` hook:

```
Success: {"data": {...}, "meta": {"request_id": "4f1c...", "duration_ms": 37}}
Failure: {"error": {"code": "AUTH_REQUIRED", "message": "Authentication required.", "detail": null}}
```

The frontend unwraps data in one place (`services/http.ts`) and turns error into an `ApiError` exception. Every screen therefore handles errors the same way. The `request_id` makes a problem traceable in the server logs.

4.6 Services: one job per file

Routes stay thin; the logic lives in `app/services/` — the equivalent of Laravel "service classes". Each file has a single responsibility, which makes the system easy to explain and to test:

Service	Responsibility
scan_service	Save any analysis result, run SOC automation, visitor claim tokens, rebuild a result for display
audit_service	Write every security-relevant event (logins, admin actions) to <code>security_events</code> . Never raises.
client_info	Real client IP, device name, IP geolocation (country, city, ISP, VPN/datacentre flags)
registration_risk	Sign-up risk score and approval decision
triage_service	Engine suggestion + confidence on reported messages; manual / assisted / auto modes
indicators + correlation_service	Extract indicators (domains, brands), group dangerous messages into incidents
threat_feed	Download and cache the OpenPhish list of live phishing URLs
domain_health	DNS / HTTPS / certificate checks of official institution domains
system_metrics	Traffic, CPU/RAM/disk, database, engine and user-experience metrics
settings_service	Platform settings stored in the database (policies, thresholds)
verification_service, otp_service, mailer, email_templates	Codes, SMTP sending, branded HTML emails
mailbox_service, inbound_service, crypto_box	Gmail/Outlook scanning, forwarded emails, token encryption
scheduler	Background jobs thread

5. The data layer: PostgreSQL and SQLAlchemy

5.1 Why PostgreSQL

- Reliable transactions and concurrency (many users scanning at once, background jobs writing at the same time).
- **JSON columns**: an analysis stores its full explanation (signals, weights, evidence, AI verdict) as JSON, a user stores the geolocation of registration — no need for dozens of extra tables.
- Partial and unique indexes (e.g. one row per mailbox message: `UNIQUE (mailbox_id, external_id)`).
- Roles and GRANTS: the application connects as **phishguard_user** with limited rights; schema changes are done as **postgres**.

5.2 The tables

Table	What one row is	Key columns
users	An account	email, password_hash (bcrypt), is_admin, status, approval_status, mfa_secret, session_version, registration_ip/geo, risk_score, quiz_xp, onboarded_at
analyses	One analysed message	user_id, text_source, score_risk, verdict, indicators (JSON explanation), urls, source (web/mailbox/forward/share), reported_at, review_label, triage_label, claim_token_hash, feedback
whitelist_domains	An official institution domain (MTN, Orange, banks...)	domain, institution, category, logo_url, website, support_contact, health (JSON)
blocked_domains	A domain treated as known-bad	domain, reason, incident_id, is_active
incidents	A scam campaign (several related messages)	ref, indicator, severity, status, analysis_ids (JSON), timeline (JSON), assigned_to
security_events	Audit trail / login history entry	event_type, severity, user_id, actor_id, ip, geo, details
platform_settings	One setting	key, value (JSON)
mailbox_connections	A connected Gmail/Outlook mailbox	provider, email, access/refresh token (encrypted), status, sync_cursor, counters
verification_challenges, mfa_sessions, oauth_accounts	Auth helpers	codes, expiry, linked Google accounts
threat_categories, suspicious_keywords, news_articles	Reference / awareness content	category texts, keywords, news

5.3 Models: tables as Python classes

```
class MailboxConnection(db.Model):                                # like: class X extends Model (Laravel)
    __tablename__ = 'mailbox_connections'
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)
    provider = db.Column(db.String(20), nullable=False)           # gmail | outlook
    access_token_enc = db.Column(db.Text)                         # encrypted, never plain
    status = db.Column(db.String(20), default='active', nullable=False)
```

Querying reads almost like Eloquent:

```
User.query.filter_by(email=email).first()                       # one row or None
Analysis.query.filter(Analysis.created_at >= since).count()     # COUNT(*)
```

```
db.session.add(obj); db.session.commit()  
db.session.rollback()
```

```
# INSERT / UPDATE  
# undo after an error
```

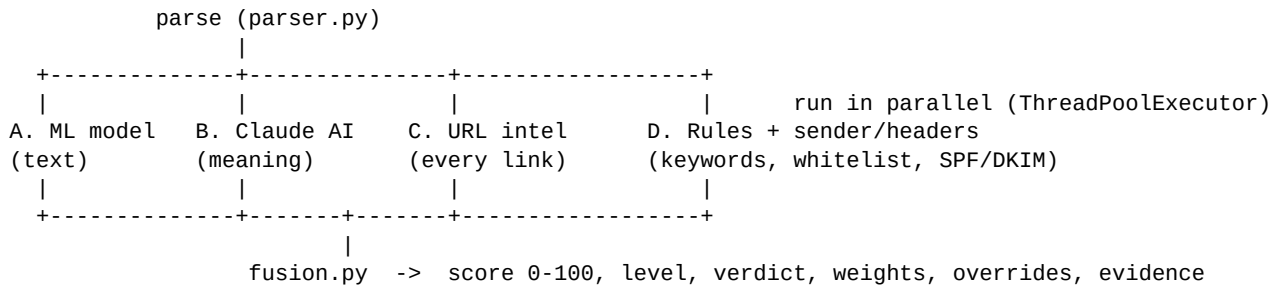
5.4 Migrations the PhishGuard way

Schema changes are numbered SQL files in `backend/database/scripts/`, applied in order in pgAdmin while connected as **postgres** to **phishguard_db**. Each script is idempotent (`ADD COLUMN IF NOT EXISTS`, `CREATE TABLE IF NOT EXISTS`) so running it twice is harmless, and ends with GRANTS for `phishguard_user`. The model in `models.py` is updated in the same change.

Script	Adds
01–03	Base schema, seed data (institutions, categories, keywords), indexes
04	Authentication upgrade: verification codes, OAuth, MFA
05–06	User dashboard and admin platform (settings...)
07	Review queue (reports, admin labels)
08	SOC platform: approval workflow, registration intelligence, audit events, incidents, blocklist, triage
09	Mailbox connections, analysis origin, visitor claim tokens, feedback, quiz progress, support contacts

6. The detection engine (the heart of the product)

Entry point: `run_pipeline(text=... | raw_eml=..., source, lang)` in `app/pipeline/__init__.py`. It takes a message and returns a score from 0 to 100 with a full explanation. The idea: **four independent opinions**, each good at something different, combined into one explained decision.



6.1 Parsing (parser.py)

Pasted text becomes a `ParsedMessage` (text + extracted URLs). An `.eml` file (a real email with headers) is parsed with Python's email package: subject, sender, reply-to, authentication results (SPF/DKIM/DMARC), attachments, text body (HTML converted to text). If the email was forwarded as an attachment, the inner (original) message is analysed, because its headers belong to the attacker. Links written without http ("mtn-verify.tk/login") are also detected.

6.2 Signal A — the ML model (30 %)

A classic text classifier trained on labelled phishing / legitimate messages: **TF-IDF** turns text into numbers (how characteristic each word is), then a **Naive Bayes** classifier (scikit-learn) outputs the probability of phishing. It is fast, works offline and never leaves the server. Trained by `train_ml.py`, saved as `ml_engine/trained/classifier.joblib`.

6.3 Signal B — Claude AI (25 %)

`ai_analyzer.py` sends the message to Anthropic's Claude with a system prompt specialised in Cameroonian scams. Three important design choices you can explain to a jury:

- **Privacy:** emails, Cameroonian phone numbers and long numbers are redacted ([EMAIL], [TEL], [NUMERO]) before anything leaves the server.
- **Prompt-injection defence:** the message is placed between `<message>` tags and the prompt says it is data, never instructions.
- **Structured output:** the answer must match a Pydantic model (classification, category, confidence, reasons, recommendation). No free text to parse; invalid answers are rejected.
- **No invented contacts:** the AI may only cite the verified CIRT-CM contact (8202, alerts@cirt.cm).
- If the key is missing or the API fails, the signal is simply absent and the other signals carry the decision.

6.4 Signal C — URL intelligence (25 %)

`url_intel.py` scores every link: domain compared with the official allowlist (**look-alikes** such as `mtn-cm.tk` or `Orange-money` are caught by string similarity and "leet" normalisation), brand words inside a foreign domain, IP addresses instead of names, suspicious TLDs (`.tk`, `.ml...`), URL shorteners, the **blocklist** fed by incidents (forces 95) and the **OpenPhish** feed of live phishing URLs.

6.5 Signal D — rules, sender and headers (20 %)

Keyword heuristics tuned for French/English scams (urgency, blocking threats, requests for PIN/codes, advance fees), plus sender checks: an **official sender** (domain in the allowlist) is reassuring; a sender **imitating** an institution, replies going to another domain, failed SPF/DKIM/DMARC or dangerous attachments raise the score.

6.6 Fusion (fusion.py)

```
weights = {"ml": 0.30, "ai": 0.25, "url": 0.25, "rules": 0.20}
# missing signals (no AI key, no links) -> their weight is shared proportionally among the others
score = sum(signal * weight)

# overrides: hard evidence beats averages
if blocklisted link or brand impersonation:    score = max(score, 85)
elif official sender + authentication passed + clean links:    score = min(score, 30)

level:    > 80 Critical    > 60 High    > 40 Medium    else Low
verdict: Critical/High -> "phishing", Medium -> "suspicious", Low -> "legitimate"
```

Why fusion instead of one model? Each signal fails differently: the ML model can be fooled by new wording, the AI can be unavailable, the URL check is blind when there is no link, rules are simple. Averaging independent opinions and adding hard overrides gives a result that is **robust and explainable** — every number shown to the user comes from one of these four boxes.

6.7 Languages

All explanations are produced in the user's language. The frontend sends Accept-Language and a lang field; pipeline/i18n.py holds the French and English sentences (tr(lang, key, **params)) and the AI is asked to answer in that language.

7. Security architecture

As a cybersecurity product, PhishGuard must itself be hard to attack. Security is layered ("defence in depth"):

Threat	Protection	Where
Stolen password database	bcrypt hashing (slow, salted); passwords never stored or logged	auth.py, user_profile.py
Password guessing	attempt counter + temporary lock, equalised response time (no user enumeration), generic messages	auth.py
Stolen password	MFA with TOTP codes (pyotp, authenticator apps); disabling MFA requires a valid code	auth.py, MfaSettings.vue
Fake / bot accounts	email code verification, disposable-email block, honeypot field, fill-time check, IP/VPN/datacentre risk score, optional admin approval	registration_risk.py
Stolen session cookie	HttpOnly + SameSite cookie; session_version (sign out everywhere); idle timeout; new-location alert	__init__.py
Abuse of an admin session	sudo mode: sensitive actions (delete user, grant admin, change key policies, unblock a domain) require the password (+MFA) again	api/security.py, SudoModal.vue
Accessing others' data	every query filters by current_user.id; admin routes use @admin_required	all blueprints
Visitors reading results / scraping	results gated behind sign-in (claim token, SHA-256 hashed, single use, 24 h); visitor rate limit per IP	scan.py, scan_service.py
Leaked mailbox tokens	OAuth only (never the password), read-only scope by default, tokens encrypted at rest, revocation on disconnect	mailbox_service.py, crypto_box.py
Prompt injection / data leak to AI	redaction + tagged input + structured output	ai_analyzer.py
Repudiation ("I didn't do it")	audit trail of logins and admin actions with IP, device, location	audit_service.py, AuditLog.vue
Secrets in code	all keys in .env (not committed)	.env / .env.example

7.1 How a login works

```
POST /api/auth/login {email, password}
-> find user, check lock, bcrypt.checkpw(password, hash)
-> account approved? email verified?
-> MFA active? yes -> respond {mfa_required: true}; frontend asks for the 6-digit code -> /api/auth/mfa
-> login_user(user) # Flask-Login writes the session cookie containing "user_id:session_version"
-> audit: login.success (IP, device, city); new country -> alert
Every next request: load_user() reads the cookie, reloads the user, checks version/status/approval.
```

8. The admin Security Operations Centre (SOC)

8.1 Report triage (80 % automation)

Users can report a message. `triage_service.py` gives each report an engine suggestion (phishing / safe) and a confidence. Three modes set in Settings: **manual** (the admin decides everything), **assisted** (reports with confidence $\geq$ threshold, default 80 %, are closed automatically; the rest wait for a human), **auto**. Automatic decisions are marked `review_source="auto"` and are excluded from the data used to retrain the models, so the engine never learns from itself.

8.2 Incidents (campaign correlation)

After each dangerous scan, `correlation_service.correlate()` extracts indicators (domains, imitated brands). When N dangerous messages (default 3) share an indicator inside a time window (default 72 h), an **incident** is opened with a severity, a JSON timeline, the list of analyses and affected users. The admin can assign it, add notes, change its status (open $\rightarrow$ investigating $\rightarrow$ contained $\rightarrow$ resolved) and **block the domain** in one click. A blocked domain is then forced to "phishing" in every future scan. Active incidents also feed the public "Scams in progress" alerts, anonymised.

8.3 Threat intelligence, allowlist, audit, system

- **Threat intel**: indicators of compromise from our own detections, cross-checked with OpenPhish, targeted brands, AI categories, blocklist.
- **Allowlist**: official domains with logos, website and hotline; live DNS/HTTPS/certificate checks (an expiring certificate on an official site is itself a risk).
- **Audit log**: every security event, filterable, exportable to CSV.
- **System**: live CPU/RAM/disk, API traffic and error rate, response times (p50/p95), database latency and size, engine components, AI coverage, background jobs and user-experience metrics.

8.4 Background jobs (scheduler.py)

Job	Every	Does
mailboxes	5 min	Scan new emails of connected mailboxes
forwarded_emails	2 min	Read the PhishGuard inbox (IMAP), analyse forwarded emails, reply
correlation	15 min	Group dangerous messages into incidents
threat_feed	1 h	Refresh the OpenPhish list
allowlist_health	12 h	Check DNS/HTTPS of official domains

It is a daemon thread started on the first request (so the Flask reloader never starts it twice). In production with several workers, this would move to a dedicated worker (Celery/RQ or a cron) — a good "future work" point.

9. Mailbox scanning, forwarding and "analyse first, sign in to see"

9.1 Connecting Gmail / Outlook (OAuth 2.0)

1. User clicks "Connect Gmail" -> POST /api/mailbox/connect/gmail
2. Backend creates a random "state" (anti-CSRF), stores it in the session, returns Google's consent URL
3. Browser goes to Google; user accepts read-only access
4. Google redirects to /api/mailbox/callback/gmail?code=...&state=...
5. Backend checks the state, exchanges the code for tokens (access + refresh), reads the address, encrypts the tokens (crypto_box) and stores a mailbox_connections row
6. First scan (last 14 days) starts in the background; then every 5 minutes

Sync: list inbox messages received after the cursor → download each as raw MIME (so real headers are analysed) → `run_pipeline(raw_eml=...)` → save with `source="mailbox"`. The unique index on (mailbox, provider message id) makes duplicates impossible. Expired access tokens are refreshed automatically; a revoked access marks the mailbox "Access removed". Safe emails keep only their subject (privacy by design).

9.2 "Forward it to PhishGuard"

For people who do not want to connect a mailbox: they forward the email to the PhishGuard address. `inbound_service.py` reads that inbox over IMAP, analyses the original email, and replies with a branded verdict email. Loop protection: automatic replies and our own address are ignored, 20 replies per sender per day.

9.3 Visitor gating

Anyone can submit a message, but the verdict is only shown after sign-in. The server stores the analysis without owner and returns only `{gated: true, claim_token}`. The browser keeps the token; after sign-in, POST /api/v2/scan/claim attaches the analysis to the account and returns the result. The token is stored hashed, works once, expires after 24 h. Visitors are also rate-limited (15 scans per hour per IP). The gate still shows a safety tip ("don't click, don't share codes") so nobody is left unprotected while deciding to sign up.

9.4 Encrypting tokens (crypto_box.py)

Tokens are encrypted before being saved. With the cryptography package it uses Fernet (AES + HMAC). Without it, a standard-library construction: an HMAC-SHA256 keystream with a random nonce, plus an HMAC tag (encrypt-then-MAC), so a modified value is detected. The key comes from `MAILBOX_ENCRYPTION_KEY`.

10. The frontend: Vue.js from a React developer's point of view

10.1 Why Vue 3 + Vite + TypeScript + Tailwind

- **Vue 3:** HTML-like templates that are easy to read for designers and juries, fine-grained reactivity (no re-render of the whole component tree), official router and store — less boilerplate than React for a dashboard-heavy app.
- **Vite:** instant dev server with hot reload, production build with code-splitting (each page is its own file, loaded when needed).
- **TypeScript:** the API shapes (AnalysisResult, MailboxConnection...) are typed, so a backend change breaks the build instead of the user.
- **Tailwind:** the same utility classes you know; dark mode driven by a .dark class on <html>.

10.2 A Single File Component, read like React

```
<script setup lang="ts">                                     // = the body of a React function component
import { computed, ref } from 'vue'
const props = defineProps<{ score: number }>()              // = function X({ score })
const emit = defineEmits<{ again: [] }>()                  // = onAgain callback prop
const open = ref(false)                                     // = const [open, setOpen] = useState(false)
const label = computed(() =>                               // = useMemo, dependencies tracked automatically
  props.score > 60 ? 'Danger' : 'OK')
function toggle() { open.value = !open.value }              // refs are read/written with .value in <script>
</script>

<template>                                                  <!-- = the JSX, but plain HTML + directives -->
  <button @click="toggle">{{ label }}</button>              <!-- onClick / {label}; no .value in template -->
  <p v-if="open">Details...</p>                             <!-- {open && <p>...</p>} -->
  <li v-for="r in reasons" :key="r">{{ r }}</li>            <!-- reasons.map(r => <li key={r}>...) -->
  <input v-model="text" />                                  <!-- value={text} onChange={e => setText(...)} -->
  <button @click="emit('again')">Again</button>             <!-- props.onAgain() -->
</template>

<style scoped> /* CSS that only applies to this component */ </style>
```

Lifecycle

onMounted(fn) = useEffect(fn, []) · onBeforeUnmount(fn) = the cleanup returned by useEffect ·
watch(source, fn) = useEffect(fn, [source]).

10.3 The layers of the frontend

```
views/ (pages) -> components/ (pieces) -> composables/ (logic) -> stores/ (Pinia state)
                                                    -> services/*.service.ts -> http.t
```

- **router/index.ts:** maps URLs to pages, lazy-loads each page, and a global guard beforeEach checks meta.requiresAuth / requiresAdmin (calls /api/auth/me).
- **services/http.ts — apiFetch():** adds Accept-Language, sends cookies, applies a timeout, unwraps {data}, converts errors to ApiError, shows an error toast, and if the server answers SUDO_REQUIRED it opens the password dialog and replays the request.
- **services/*.service.ts:** one file per API domain (analysis, auth, admin, soc, mailbox, experience, public...). Components never call fetch directly.
- **stores/ (Pinia):** shared state. auth (current user, login modal), analysis (current result, visitor gate, offline queue), notifications (toasts), user dashboard data...

- **composables/**: reusable logic — useAnalysis, useAuth, useTheme (dark mode), useSpeech (read aloud), useOcr (screenshot reading).
- **i18n/**: our own tiny translation system: t('key', {params}), dictionaries per language and domain, fallbacks (Pidgin→English→French; Ewondo/Fulfulde→French).

10.4 A store, read like Redux/Zustand

```
export const useAnalysisStore = defineStore('analysis', () => {
  const result = ref<AnalysisResult | null>(null)      // state
  const gate = ref<GatedScan | null>(null)            // visitor: result locked
  const isAnalyzing = ref(false)
  async function analyze(text: string) {              // action
    isAnalyzing.value = true
    const outcome = await analysisService.analyzeMessage(text)
    if (isGated(outcome)) gate.value = outcome else result.value = outcome
    isAnalyzing.value = false
  }
  return { result, gate, isAnalyzing, analyze }
})
// in a component: const store = useAnalysisStore(); store.analyze(text)
```

10.5 Installable app (PWA)

public/manifest.webmanifest makes the site installable on a phone and declares a **share target**: from WhatsApp or SMS, "Share → PhishGuard" opens /check?text=... and the check starts automatically. public/sw.js (service worker, production only) caches the app shell so it opens offline; API calls are never cached. A message typed offline is queued and analysed when the network returns.

11. Page by page: what you see and where it comes from

11.1 Public pages

Page (URL)	What the user sees	Data / endpoint	Main files
Home (/)	Hero, the message checker, live scam alerts, threat categories, threat map, about	/api/v2/scan, /api/public/alerts, /api/threats/*	HomeView, Hero, Analyzer, CommunityAlerts, Categories, ThreatIntel
Check (/check)	The checker alone with 3-step guide; also opened by Share, email links (?claim, ?analysis)	/api/v2/scan, /scan/claim, /scan/<id>	CheckView, Analyzer, components/check/*
Learn (/learn)	"Real or fake?" quiz, XP level and badges, golden rules	/api/user/quiz/answer, /api/user/summary	LearnView, data/quiz.ts
Privacy (/privacy)	What is stored and why; delete my history	DELETE /api/user/analyses	PrivacyView
Login / Register (modal)	Soft forms with logo; register = full name, region, city, terms; email code; MFA code if enabled	/api/auth/register, /login, /mfa/verify	AuthModal, LoginForm, RegisterForm, OtpVerificationForm

11.2 The checker, component by component

Component	Role
ScanComposer	Text area; buttons Paste, Screenshot (Tesseract.js reads the image in the browser), .eml file; examples; Check button
ScanProgress	The five steps lit up with icons (read, sender, links, AI, verdict) while the server works
ResultGate	For visitors: "Your result is ready — sign in", perks, safety tip. After sign-in the result appears automatically
ResultCard	Verdict emblem with score ring, one-sentence summary, read-aloud, report, new check
WhatToDo	Action cards for the verdict + the imitated institution's official website/hotline + "I already clicked" steps
HighlightedMessage	The message with dangerous links, code requests, urgency words, money bait and phone numbers highlighted, with a legend
ThreatAlert	Pop-up alert when the verdict is dangerous or suspicious

11.3 User space

Page	What the user sees	Endpoint(s)
Dashboard > Home	Vigilance gauge, today's KPIs, monthly summary (scams avoided, level), shortcuts (mailboxes, quiz, privacy), recent analyses, tip, live alerts, MFA reminder, welcome tour on first visit	/user/dashboard/overview, /user/summary, /public/alerts, /user/onboarded
Dashboard > Analyze	The checker, embedded	/v2/scan
Dashboard > History	All analyses with search, verdict and origin filters, delete one; opens a detail drawer	/user/analyses, /user/analyses/<id>
Dashboard > Notifications	Threats found, confirmed reports, tips	/user/notifications
Dashboard > Settings	Profile, avatar, password, MFA	/user/profile, /auth/mfa/*
My mailboxes (/dashboard/mailboxes)	Connected Gmail/Outlook, counters, scan now, pause, disconnect, forwarding address, privacy	/mailbox, /mailbox/connect, /mailbox/<id>/sync

11.4 Admin console (/admin)

Page	What the admin sees and does	Endpoint(s)
Overview	Global KPIs, traffic chart, threat map, SOC band (system, incidents, triage, pending accounts, blocklist)	/admin/stats..., /admin/soc/summary
User management	Accounts with risk score, registration IP/city/ISP/VPN flags, device; approve / ask more verification / reject; suspend, unlock, force logout, role; activity	/admin/users...
Reports (review queue)	Reported and borderline messages with engine suggestion + confidence; decide phishing/safe; bulk accept; triage mode	/admin/reviews...
Allowlist	Official domains with logos, website, hotline; live health; add/edit/disable/delete	/admin/whitelist...
Incidents	Campaigns with severity, timeline, checklist, notes, assignment, block domain	/admin/incidents...
Threat intelligence	Indicators, OpenPhish matches, targeted brands, AI categories, blocklist	/admin/intel..., /admin/blocklist
System	Status, CPU/RAM/disk, traffic, DB, engine, user-experience metrics, background jobs	/admin/system
Audit log	All security events, filters, CSV export	/admin/security/events
Settings	Admin profile, security centre (posture score A–D, MFA, sessions, login history), platform policies	/admin/settings, /admin/security/*
ML models, Integrations	Demonstration screens (sample data, marked as demo)	—

12. Three journeys, end to end

12.1 A visitor checks an SMS, then signs in

1. /check ScanComposer (v-model draft) -> click "Check"
2. analysis.store.analyze(text) -> analysisService.analyzeMessage -> apiFetch POST /api/v2/scan
3. Flask: before_request (request id, timer) -> scan.py: visitor? rate limit by IP
4. run_pipeline(): parse -> ML | Claude | URL intel | rules (parallel) -> fusion -> score 91.9, "Critical"
5. scan_service.finalize() -> verdict "phishing"; save() -> INSERT INTO analyses (user_id NULL)
post_scan(): triage if borderline, correlate() -> maybe opens/extends an incident
6. visitor -> issue_claim(): random token, SHA-256 stored, 24 h -> JSON {gated, claim_token, checks}
7. Browser: store.gate = ..., token saved in localStorage -> ResultGate shown
8. User signs in (AuthModal -> /api/auth/login -> cookie set) -> Analyzer watches auth.user
9. store.claim() -> POST /api/v2/scan/claim -> analysis.user_id = me, token erased -> full result
10. ResultCard + WhatToDo + HighlightedMessage render; ThreatAlert pops up

12.2 A phishing email arrives in a connected Gmail

scheduler (every 5 min) -> mailbox_service.sync_all_due -> sync(conn)
-> access_token() (decrypt; refresh if expired) -> Gmail API list messages after cursor
-> for each: download raw MIME -> run_pipeline(raw_eml) -> save(source="mailbox", keep body only if danger)
-> counters updated, optional "PhishGuard" label, audit event mailbox.threats
-> correlate(): 3rd email with the same fake domain -> INCIDENT opened
User: notifications show "Dangerous email in your mailbox"; History shows it with the inbox icon.
Admin: SOC band shows the new incident -> Incidents page -> "Block domain"
-> blocked_domains row -> every next scan containing that domain is forced to >= 85.

12.3 An admin deletes a user (sudo mode)

UserManagement.vue -> adminService.deleteUser(id) -> DELETE /api/admin/users/<id>
Flask: @admin_required OK, @sudo_required -> no recent re-authentication -> 403 SUDO_REQUIRED
apiFetch sees SUDO_REQUIRED -> SudoModal asks password (+ MFA) -> POST /api/admin/security/sudo
-> session["sudo_until"] = now + N minutes -> apiFetch replays DELETE -> success
-> audit_service.record("admin_user_deleted") -> visible in Audit log

13. Reading a result: every output explained

Output	Meaning	Produced by
score (0–100)	Weighted combination of the available signals, after overrides	fusion.py
level	Critical > 80, High > 60, Medium > 40, else Low	fusion.verdict_for
verdict	phishing (Critical/High), suspicious (Medium), legitimate (Low) — shown as Dangerous / Suspicious / Safe	scan_service.finalize
signals	Each signal's own score: ml, ai, url, rules (null = unavailable)	pipeline
weights	Weight actually used for each available signal (redistributed if one is missing)	fusion.py
overrides	Sentence explaining a hard rule (e.g. "brand impersonation: raised to 85")	fusion.py
evidence + evidence_positive	List of reasons; positive = reassuring (official sender) shown in green	rules, url_intel, AI
ai.category / confidence	Type of scam (mobile_money_fraud, job_scam...) and the AI's confidence	ai_analyzer.py
recommendation	Plain-language advice (AI, or default CIRT-CM advice)	ai_analyzer / i18n
brand + official	Institution being imitated and its official website/hotline from the allowlist	sender.py, url_intel, scan_service.official_contact
urls	Every link with its own score, blocklisted flag and domain status (used for highlighting)	url_intel.py
duration_ms	Time the pipeline took	pipeline
source	web, mailbox, forward or share	scan.py / mailbox / inbound

Example to explain orally. “MTN MoMo: your account will be suspended in 24h. Confirm your PIN at <http://momo-mtn-verif.tk>” → ML sees phishing vocabulary; Claude classifies mobile_money_fraud at 97 %; URL intel flags a .tk domain naming MTN (brand impersonation); rules find urgency + a PIN request. Fusion gives ~92, the brand-impersonation override guarantees at least 85 → Critical → Dangerous. WhatToDo shows “don't click / never give a code / report”, plus MTN's official website from the allowlist.

14. Running, testing and deploying

14.1 Daily commands

```
# Backend (terminal 1)
cd backend
venv\Scripts\activate
pip install -r requirements.txt
python run.py                                # http://localhost:5000 (auto-reload on file change)

# Frontend (terminal 2)
cd frontend/PhishGuard-AI
npm install
npm run dev                                  # http://localhost:5173
npm run build                                # type-check (vue-tsc) + production build into dist/

# Tests
pytest                                       # backend unit tests
```

14.2 Configuration (.env)

DATABASE_URL, SECRET_KEY, FRONTEND_URL, CORS_ORIGINS; SMTP_* (emails);
GOOGLE_CLIENT_ID/SECRET (Google login + Gmail); MICROSOFT_CLIENT_ID/SECRET (Outlook);
MAILBOX_ENCRYPTION_KEY; INBOUND_IMAP_* (forwarding); ANTHROPIC_API_KEY (AI). Never
commit it; rotate a key as soon as it has been exposed.

14.3 Going to production (what you would say to a jury)

- Serve the built frontend (dist/) and proxy /api to Flask behind **HTTPS** (nginx); run Flask with **gunicorn** (already in requirements).
- Set SESSION_COOKIE_SECURE=True, a strong SECRET_KEY, real SMTP domain with SPF/DKIM/DMARC so emails avoid spam.
- Move the scheduler to a dedicated worker; add Flask-Limiter for global rate limiting; backups of PostgreSQL.
- For Gmail to the general public: Google app verification + security assessment (restricted scope).

15. Questions a jury may ask — and how to answer

Q. Why combine four signals instead of just using AI?

- AI can be unavailable, costly or manipulated; ML is fast and offline; URL analysis catches look-alike domains the text hides; rules encode local knowledge. Fusion with redistributed weights keeps working when one fails, and overrides make hard evidence decisive. Every point of the score is explainable.

Q. How do you protect user privacy when sending messages to an AI?

- Personal data (emails, phone numbers, long numbers) is redacted before sending; only the text needed for the analysis is sent; mailbox scanning stores only subjects for safe emails; users can delete their history in one click.

Q. How do you prevent prompt injection?

- The message is enclosed in tags and declared as data; the output must match a strict schema (Pydantic); the AI cannot trigger actions — it only returns an opinion that is one of four signals.

Q. Why sessions instead of JWT?

- Same-site web app: an HttpOnly cookie cannot be read by JavaScript (XSS-resistant), and session_version lets us revoke all sessions instantly — revoking a JWT needs extra infrastructure.

Q. What happens if an admin account is hijacked?

- Sensitive actions require re-authentication (sudo) with MFA; every action is in the audit log with IP and location; sessions can be revoked everywhere; the security centre scores the posture.

Q. Why must visitors sign in to see results?

- It protects the service against automated abuse (scraping verdicts to tune scams), builds a history for the user and the SOC, and still shows a safety tip immediately so no one is left exposed.

Q. How do incidents work?

- Indicators from dangerous analyses are correlated: N messages sharing a domain or an imitated brand within a time window open an incident with a timeline; the admin can block the domain, which then forces future scans to phishing.

Q. How does the triage reach 80 % automation without harming quality?

- Only high-confidence engine suggestions are closed automatically (threshold configurable), accuracy is measured against human decisions, and automatic labels are excluded from retraining data.

Q. How is the mailbox connection secure?

- OAuth 2.0 (password never seen), read-only scope by default, CSRF state check, encrypted tokens, revocation on disconnect, deduplication, privacy-preserving storage.

Q. Why Vue and Flask?

- Vue: readable templates, fine-grained reactivity, official router/store — fast to build dashboards. Flask: minimal API framework in the language of the ML ecosystem, so models and API share one codebase.

Q. What are the limits and future work?

- Background jobs in a dedicated worker, global rate limiting, retraining on the human-labelled export, Google verification for public Gmail scanning, native mobile app with SMS reading, more complete local-language translations.

16. Glossary

Term	Definition
API / endpoint	A URL the frontend calls to get or change data (e.g. POST /api/v2/scan).
Blueprint	Flask group of routes with a common URL prefix.
ORM	Library mapping tables to classes (SQLAlchemy here, Eloquent in Laravel).
Migration	A script changing the database structure (numbered SQL files here).
Session cookie	Small HttpOnly cookie identifying the signed-in user.
bcrypt	Slow, salted password hashing algorithm.
MFA / TOTP	Second factor: 6-digit code from an authenticator app, changing every 30 s.
Sudo mode	Short period after re-entering the password during which sensitive admin actions are allowed.
OAuth 2.0	Standard to let an app access an account (Gmail) with the user's consent, without their password.
TF-IDF / Naive Bayes	Text-to-numbers representation / probabilistic classifier used by the ML signal.
LLM	Large language model (Claude) used for semantic analysis.
Look-alike domain	Domain imitating a real one (Orange-money.com, mtn-cm.tk).
IOC	Indicator of compromise: domain, URL, sender linked to an attack.
SOC	Security Operations Centre: team/tools monitoring and responding to threats.
Incident	A correlated campaign of related malicious messages.
Triage	Sorting reports by likely outcome and confidence.
OpenPhish	Public feed of currently active phishing URLs.
SPF / DKIM / DMARC	Email authentication checks proving the sender domain is genuine.
OCR	Reading text from an image (Tesseract.js, in the browser).
PWA	Progressive Web App: installable website with offline support and share target.
Pinia / store	Vue's shared state container (like Redux/Zustand).
Composable	Reusable Vue logic function (like a React custom hook).
Proxy (Vite)	Dev server forwarding /api requests to Flask.